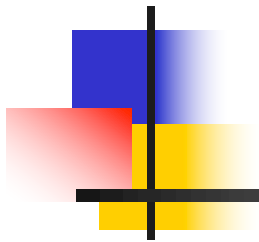
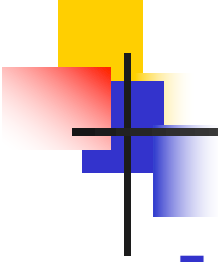


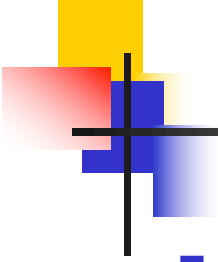
Tema 1: Introducción al Desarrollo de Aplicaciones con Frameworks Modernos





Índice

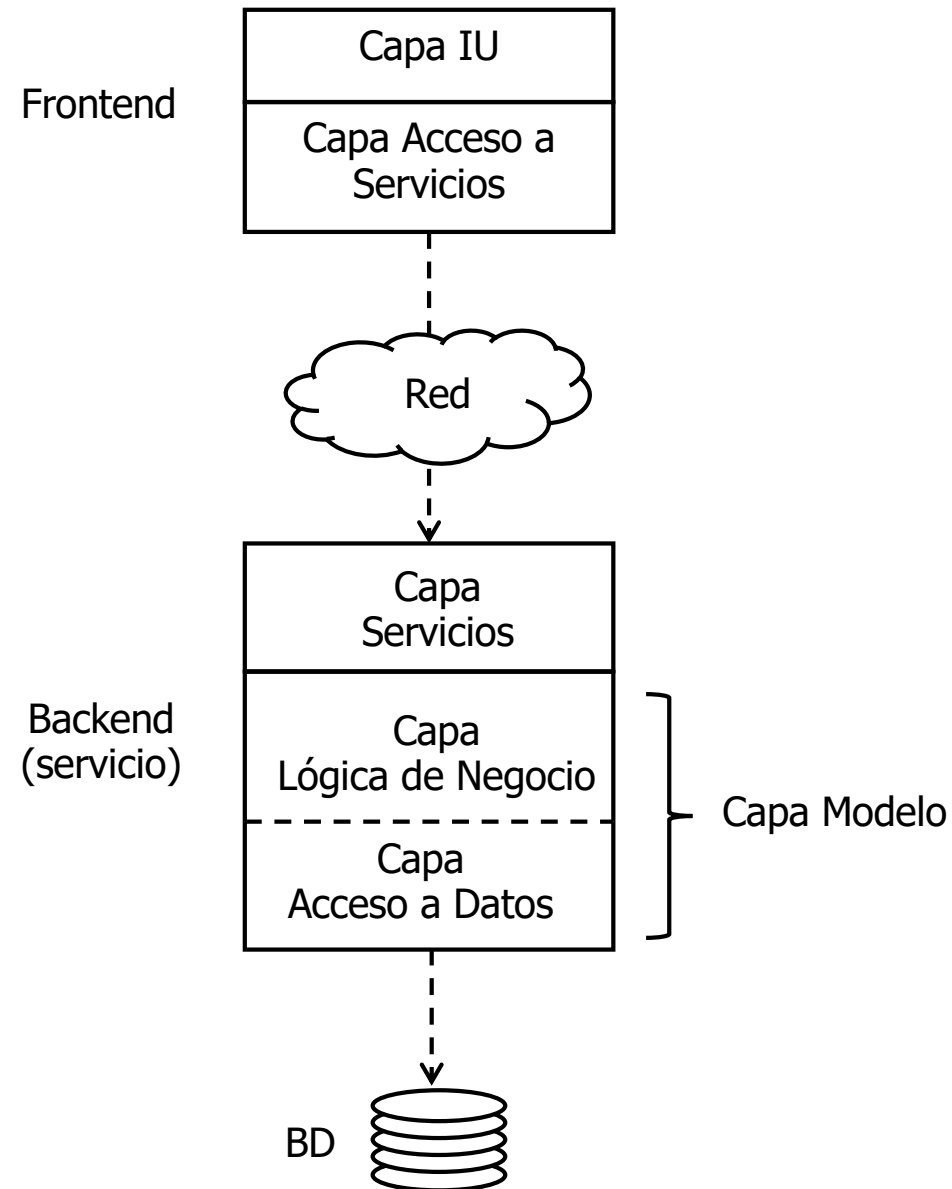
- Arquitecturas típicas de aplicaciones
- Entornos de desarrollo
- Nuestro entorno de desarrollo para aplicaciones web SPA
- Caso de estudio usado como hilo conductor
- Visión global del enfoque de desarrollo que seguiremos



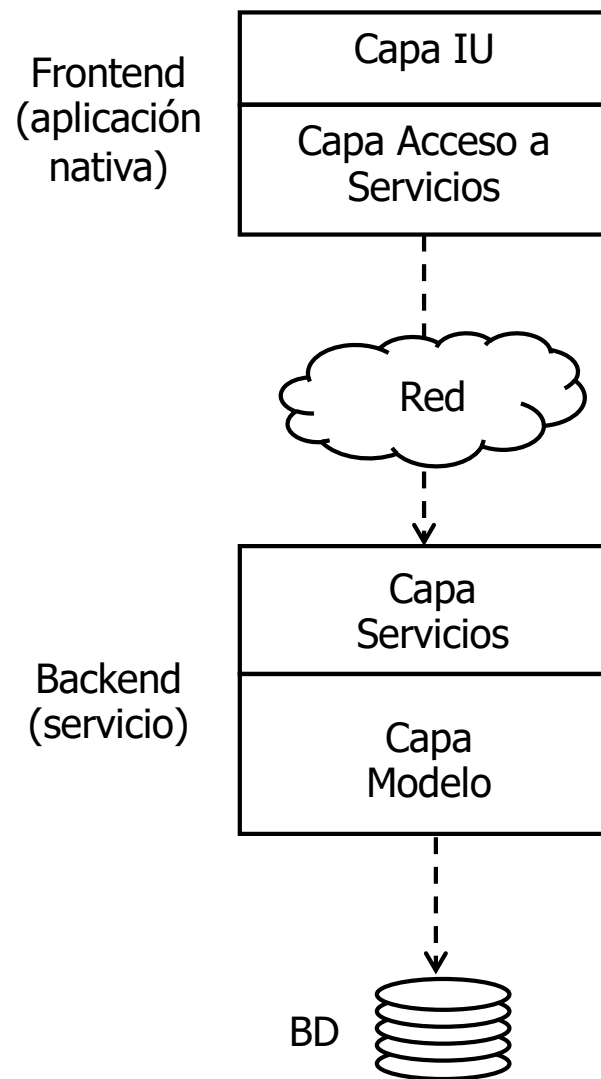
Arquitecturas típicas de aplicaciones

- Arquitectura de referencia
- Aplicación nativa (desktop o móvil)
- Aplicación web del lado servidor con capa Modelo local
- Aplicación web del lado servidor con capa Modelo remota
- Aplicación web SPA (Single Page Application)

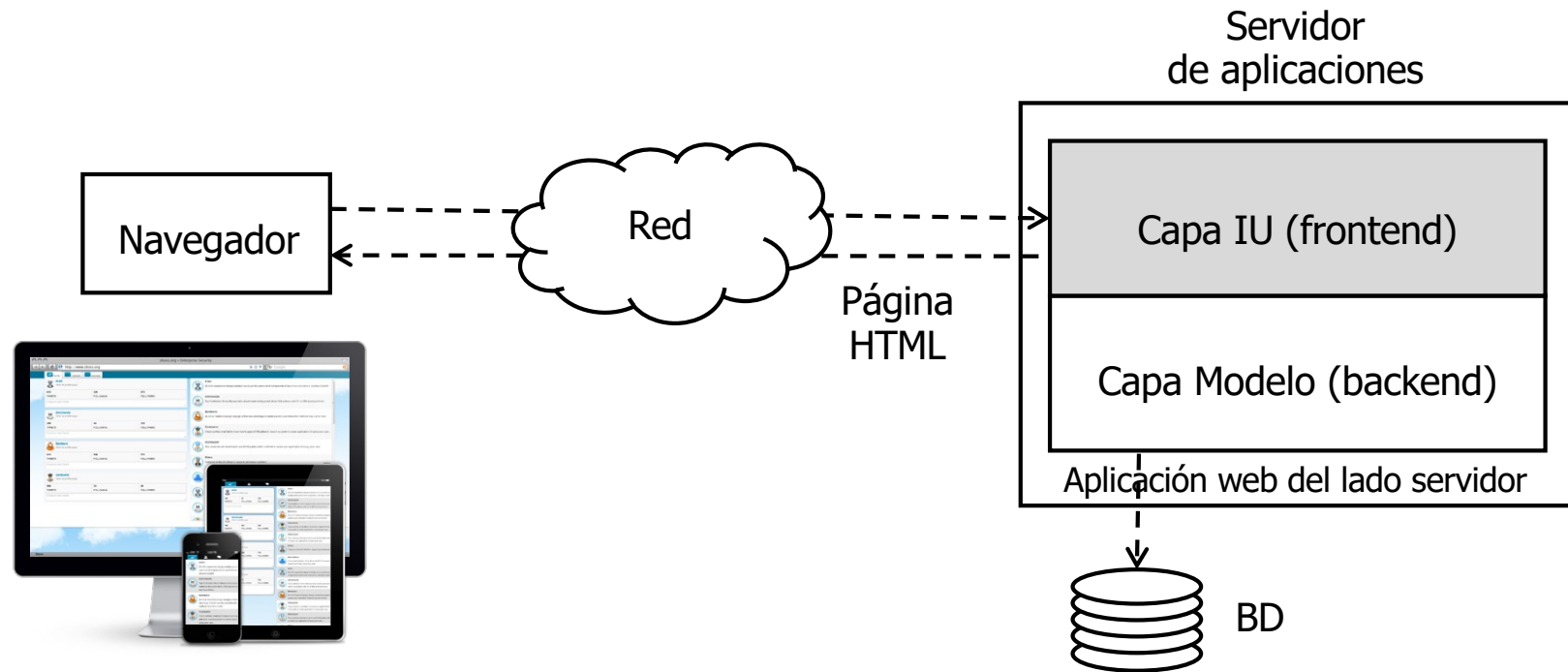
Arquitectura de referencia



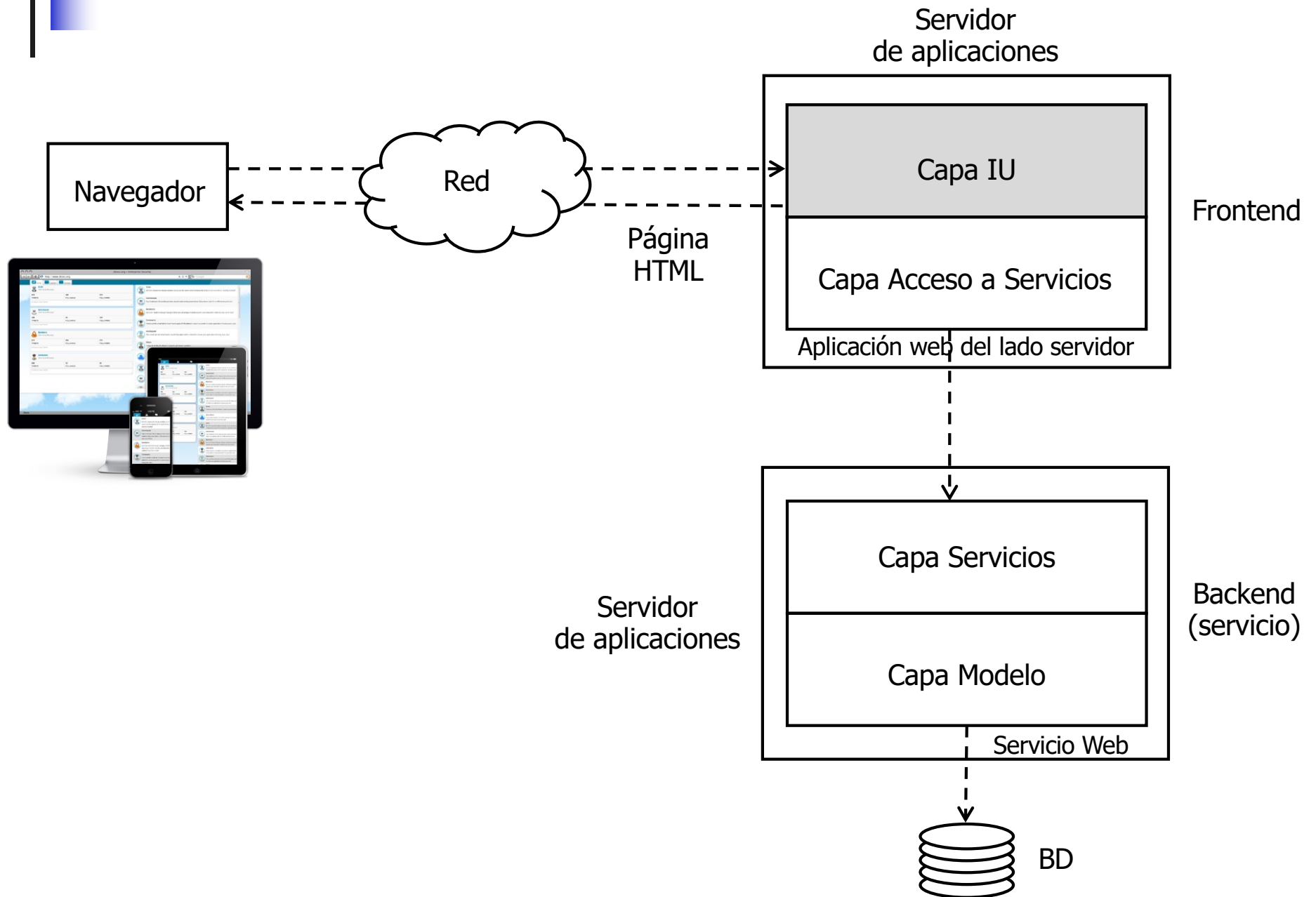
Aplicación nativa (desktop o móvil)



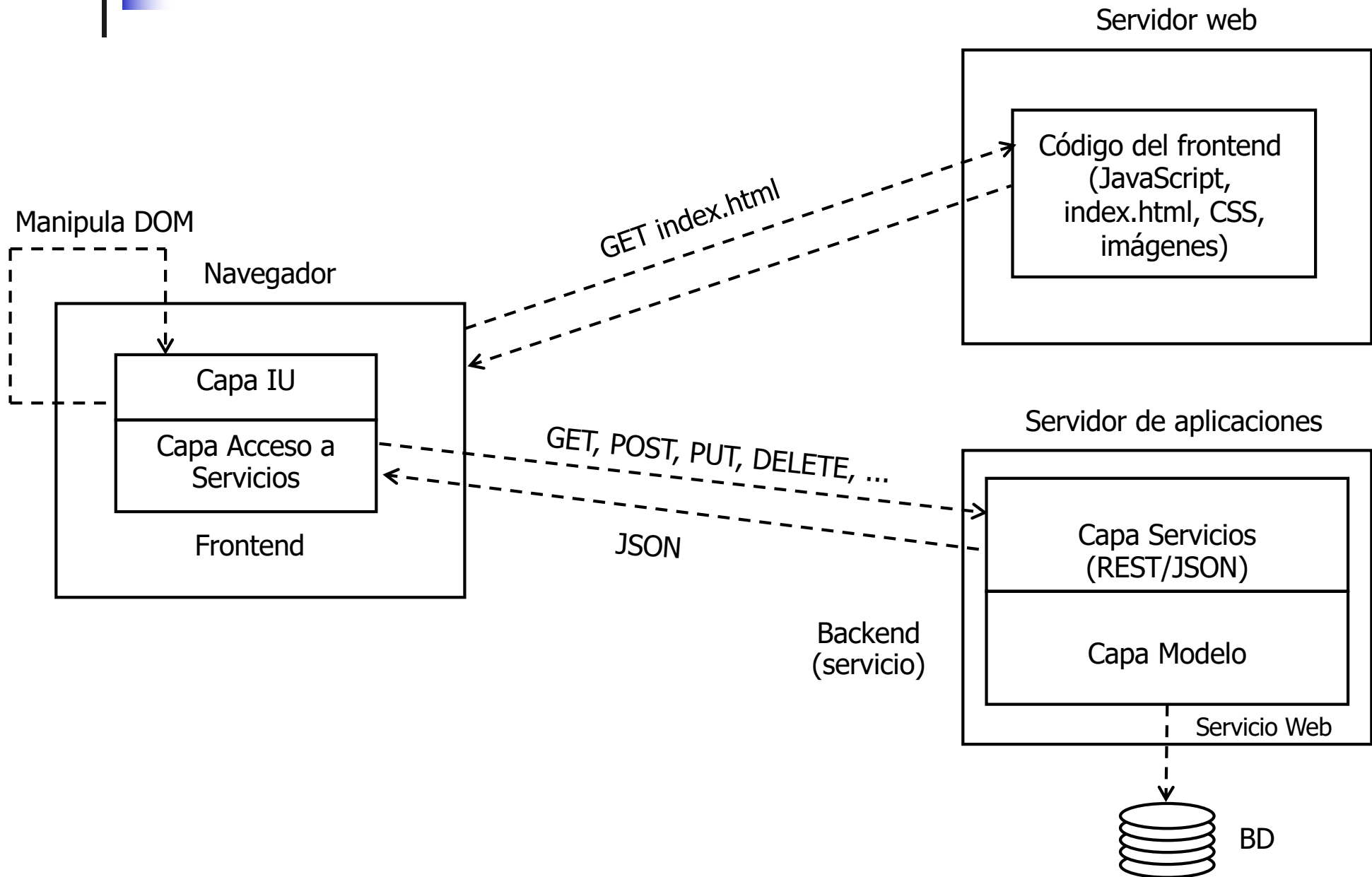
Aplicación web del lado servidor con capa Modelo local

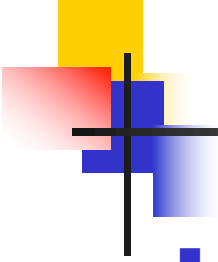


Aplicación web del lado servidor con capa Modelo remota



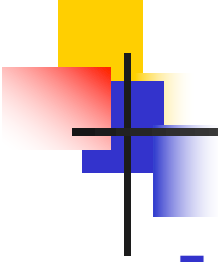
Aplicación web SPA





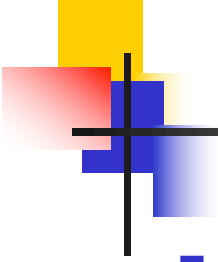
Aplicación web SPA

- Una aplicación SPA (Single Page Application) es una aplicación web que consta de
 - Un frontend JavaScript que se ejecuta en el navegador
 - Un backend (servicio típicamente REST/JSON + capa Modelo) al que accede el frontend
- Cuando el usuario introduce la URL de la aplicación en el navegador, éste envía una petición HTTP GET, cuya respuesta es una página HTML sencilla (e.g. `index.html`), que incluye referencias al código JavaScript
- Se descarga el código JavaScript, se ejecuta y se pinta la IU
- La IU del frontend reacciona a los eventos causados por el usuario (e.g. clics en enlaces y botones)
 - Realiza una petición al backend a través de la capa Acceso a Servicios (e.g. búsqueda de productos)
 - Manipula el árbol DOM de la página que muestra el navegador usando los datos devueltos por el servicio (e.g. construye una tabla con los resultados de la búsqueda y la inserta en el árbol DOM)



Entornos de desarrollo

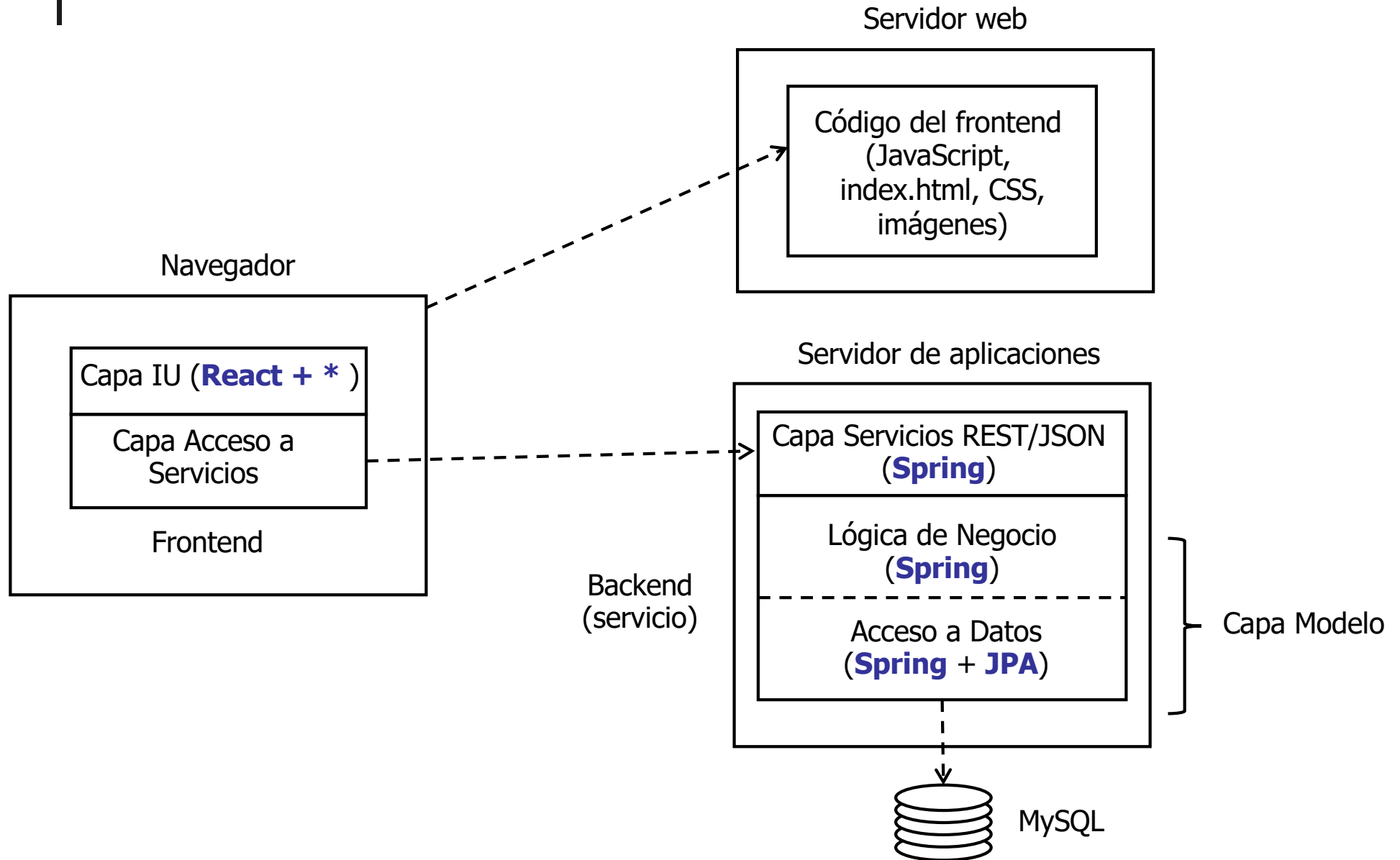
- Backend y aplicaciones web del lado servidor
 - Java
 - Jakarta EE
 - Ecosistema Spring: <https://spring.io>
 - .NET: <https://dotnet.microsoft.com>
 - Ruby on Rails: <https://rubyonrails.org>
 - Frameworks para PHP (e.g. Laravel: <https://laravel.com>)
 - Frameworks para Node (e.g. Express: <https://expressjs.com>)



Entornos de desarrollo

- Frontends de aplicaciones web SPA
 - Angular: <https://angular.dev/>
 - React + ecosistema de librerías: <https://react.dev/>
 - Vue.js + ecosistema de librerías: <https://vuejs.org>
- Aplicaciones nativas para desktop y móvil
 - Los SDKs nativos de los sistemas operativos para desktop y móvil permiten desarrollar aplicaciones nativas con las máximas capacidades posibles
 - Alternativamente, existen soluciones cross-platform, que a partir de una única distribución fuente generan ejecutables para distintos sistemas operativos
 - Se comentan en el tema 8

Nuestro entorno de desarrollo para aplicaciones web SPA

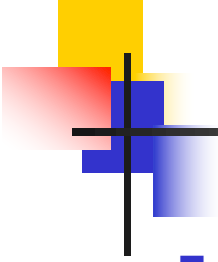


- [Java] Backend

- Spring proporciona diversos frameworks para implementar ágilmente un backend en Java
 - Por comodidad, nos referiremos a todos ellos como Spring
- Para el acceso a bases de datos relacionales usaremos **JPA** (Java Persistence API)
 - API estándar de un mapeador objeto-relacional
 - La implementación de un mapeador objeto-relacional internamente hace uso de JDBC

- [JavaScript] Frontend

- **React**
- Adicionalmente, usaremos algunos frameworks famosos del ecosistema React



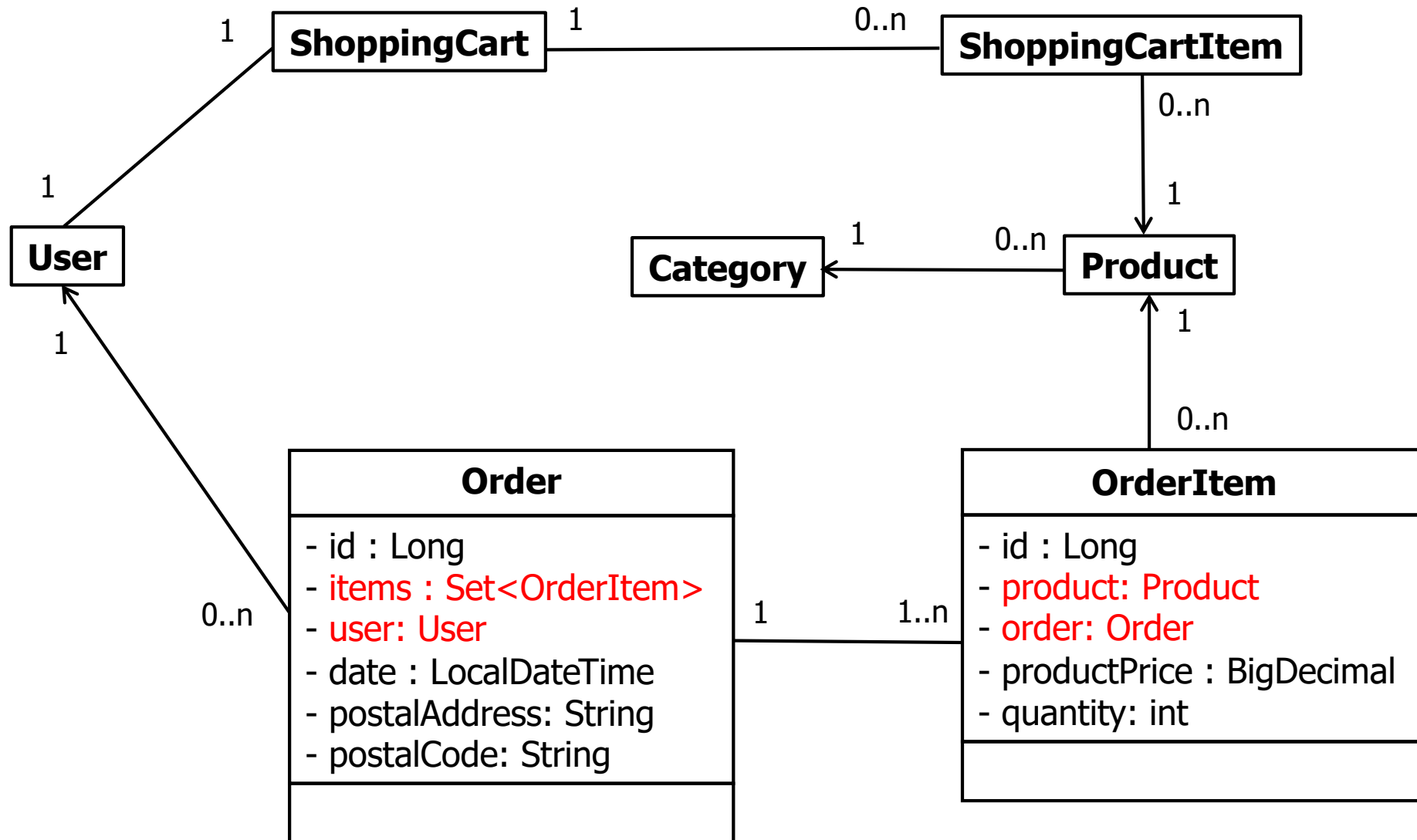
Caso de estudio usado como hilo conductor

- Se explicará el desarrollo de cada capa tomando como referencia un ejemplo de una sencilla tienda de comercio electrónico (**PA Shop**)
 - Registro de usuarios
 - Búsqueda de productos
 - Gestión del carrito
 - Compra
 - Visualización de pedidos
- Demo

Visión global del enfoque de desarrollo que seguiremos

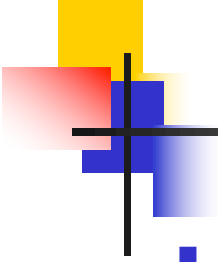
[Backend] Capa Acceso a Datos (capa Modelo)

■ Entidades



- Entidades (cont)

- Usaremos las anotaciones de la API estándar de JPA para especificar cómo mapear las entidades a la base de datos
 - Cada entidad se mapea a una tabla
 - Cada propiedad se mapea a una columna
- Las relaciones se modelan de manera natural, en lugar de usar claves foráneas en las entidades
 - **Order** no tiene el atributo **userId (Long)**, sino **user (User)**

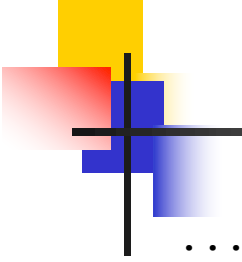


Visión global del enfoque de desarrollo que seguiremos [Backend] Capa Acceso a Datos (capa Modelo)

- DAOs

```
public interface OrderDao extends  
    CrudRepository<Order, Long> {  
  
    Slice<Order> findByUserIdOrderByDateDesc(Long userId,  
        Pageable pageable);  
  
}
```

- Especificaremos las interfaces de los DAOs
- Spring implementa automáticamente los métodos CRUD (**CrudRepository**)
- Spring es capaz de implementar automáticamente métodos de búsqueda avanzada cuando se siguen convenciones de nombrado (e.g. **findByUserIdOrderByDateDesc**)

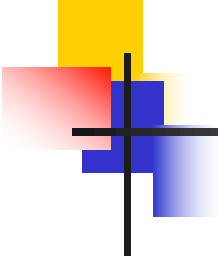


Visión global del enfoque de desarrollo que seguiremos [Backend] Capa Lógica de Negocio (capa Modelo)

...

@Transactional

```
public class ShoppingServiceImpl implements ShoppingService {  
    ...  
    @Override  
    public Order buy(Long userId, Long shoppingCartId,  
        String postalAddress, String postalCode)  
        throws InstanceNotFoundException, PermissionException,  
        EmptyShoppingCartException {  
  
        ShoppingCart shoppingCart = permissionChecker.  
            checkShoppingCartExistsAndBelongsTo(shoppingCartId,  
                userId);  
  
        if (shoppingCart.isEmpty()) {  
            throw new EmptyShoppingCartException();  
        }  
  
        Order order = new Order(shoppingCart.getUser(),  
            LocalDateTime.now(), postalAddress, postalCode);  
        orderDao.save(order);  
    }  
}
```



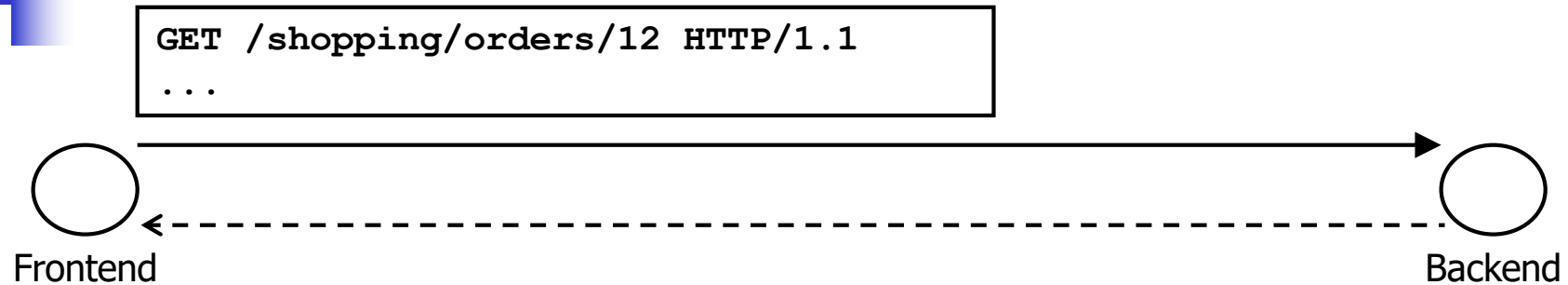
Visión global del enfoque de desarrollo que seguiremos [Backend] Capa Lógica de Negocio (capa Modelo)

```
for (ShoppingCartItem shoppingCartItem :  
    shoppingCart.getItems()) {  
  
    OrderItem orderItem = new OrderItem(  
        shoppingCartItem.getProduct(),  
        shoppingCartItem.getProduct().getPrice(),  
        shoppingCartItem.getQuantity());  
  
    order.addItem(orderItem);  
    orderItemDao.save(orderItem);  
    shoppingCartItemDao.delete(shoppingCartItem);  
  
}  
  
shoppingCart.removeAll();  
  
return order;  
  
}  
  
}
```

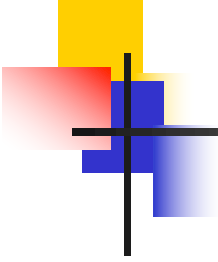
- Usaremos un enfoque **declarativo** para especificar la transaccionalidad de los métodos de los servicios que ofrece la capa Lógica de Negocio
- La anotación `@Transactional` a nivel de clase declara que todos los métodos del servicio son transaccionales
 - En la implementación de `buy`, todas las operaciones de los DAOs se ejecutan dentro de la misma transacción
 - Si la ejecución de `buy` no lanza ninguna excepción, se termina con `commit`
 - Si se lanza una excepción checked, se termina con `commit`
 - Si se lanza una excepción de runtime o un `Error`, se termina con `rollback`

Visión global del enfoque de desarrollo que seguiremos

[Backend] Capa Servicios



```
HTTP/1.1 200 OK
...
{
  "id": 12,
  "items": [
    {
      "id": 24,
      "productId": 1,
      "productName": "2001: A Space Odyssey [Blu-ray]",
      "productPrice": 11.99,
      "quantity": 1
    },
    ...
  ],
  "date": "2024-03-08T13:18:50",
  "totalPrice": 31.63,
  "postalAddress": "Rue del Percebe, 13",
  "postalCode": "12345"
}
```



Visión global del enfoque de desarrollo que seguiremos [Backend] Capa Servicios

...

```
@RequestMapping("/shopping")
```

```
public class ShoppingController {
```

...

```
@GetMapping("/orders/{orderId}")
```

```
public OrderDto findOrder(@RequestAttribute Long userId,
```

```
    @PathVariable Long orderId)
```

```
    throws InstanceNotFoundException, PermissionException {
```

```
    return toOrderDto(shoppingService.findOrder(userId, orderId));
```

```
}
```

--► Se convierte automáticamente a JSON

```
}
```

- Una función por cada caso de uso que se puede invocar remotamente

```
export const findOrder = async orderId =>  
  await appFetch('GET', `/shopping/orders/${orderId}`);
```

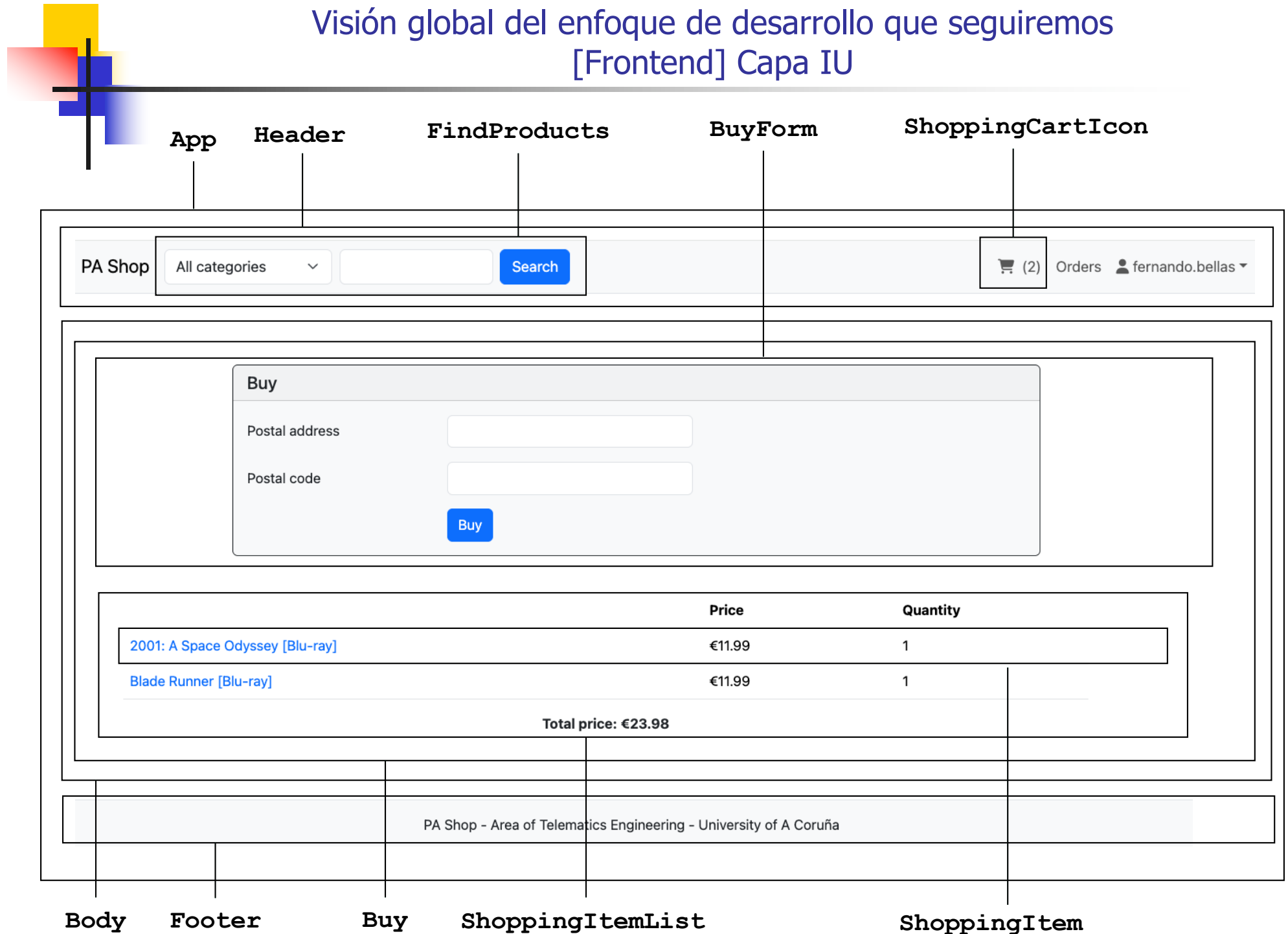
- Ejemplo de invocación desde la capa IU

```
const response = await backend.shoppingService.findOrder(orderId);  
  
if (response.ok) {  
  doSomethingWithResult(response.payload);  
} else {  
  doSomethingWithErrors(response.payload);  
}
```

- La implementación de la IU girará en torno al uso de la librería React + algunas librerías populares del ecosistema React
- Enfoque orientado a componentes
 - El desarrollador implementa la IU como la composición de componentes pequeños
 - Cada componente genera un trozo de HTML y responde a los eventos del usuario
- Ventajas
 - Divide y vencerás: se aborda el desarrollo de una IU compleja mediante la implementación de componentes pequeños
 - Algunos componentes serán reusables, dentro de una aplicación, e incluso entre aplicaciones

Visión global del enfoque de desarrollo que seguiremos

[Frontend] Capa IU



■ Ejemplo de componente

```
...  
const Buy = () => {  
  ...  
  return (  
    <>  
      <BuyForm shoppingCartId={cart.id}/>  
      <ShoppingItemList list={cart}/>  
    </>  
  );  
}  
...
```

- **BuyForm** genera el HTML correspondiente al formulario de compra
- **ShoppingItemList** genera el HTML correspondiente al carrito de la compra